

National College of Ireland
MSc in Cloud Computing — Semester 2

Scalable Cloud Programming
(MSCCLOUD_JAN26BI)
Continuous Assessment (CA)

Module Lecturers: Arjun Chikkankod, Aqeel Kazmi

Release Date: 22nd June 2026

Due Date: 4th August 2026, 5:00 pm

Weighting: 50% of the module (marked out of 100)

Assessment Period: Semester 2 — May–August 2026 (Jan 2026 cohort)

Instructions

- Students must work in **groups of two**.
- You are to design, implement, and report on a **Python-based scalable cloud computing project** using the **AWS Academy Learner Lab**.
- All submissions must be presented in **IEEE double-column format** (maximum 10 pages).
- Upload your code to the **GitHub repository** and include the link in the report.
- Upload the presentation video to the **OneDrive** location provided by NCI or YouTube and include the link in the report.
- The report must include project objectives, tools used, implementation architecture, results, and critical analysis.

Submission Details

- Upload the project report in **PDF format** to the Moodle link provided.

Late submissions are subject to a late submission penalty except where a student has applied for and been granted an extension through NCI360.

Turnitin: All report submissions are electronically screened for evidence of academic misconduct (plagiarism and collusion).

Project Description

In this project you will design and implement a **scalable, cloud-based real-time analytics system** that ingests a continuous stream of data, processes it through **both a batch layer and a speed layer** (a Lambda architecture), and scales elastically under varying load. You will use **Python and Apache Spark** for processing and **AWS services** for ingestion, deployment, and storage.

You may use **any public streaming or stream-able dataset**. Verified free sources that need no API key:

- **Wikimedia Event Streams** — a true real-time push stream (Server-Sent Events) of every edit across Wikipedia and sister projects: <https://stream.wikimedia.org/v2/stream/recentchange>. The cleanest livestream option, consumable with any HTTP client.
- **USGS Earthquake feeds** — official real-time GeoJSON feeds updated every minute, e.g. https://earthquake.usgs.gov/earthquakes/feed/v1.0/summary/all_hour.geojson. Polled and replayed into your pipeline at a controlled rate.
- **Open Notify ISS position / SpaceX API** — free, no-auth feed for lighter pipelines.
- **Replayed log or clickstream data** — take any public log/clickstream dataset and replay it at a controlled rate to simulate a live stream.
- **Propose your own** — a crypto-exchange public WebSocket (e.g. a major exchange public market-data channel), a transit GTFS-Realtime feed, an air-quality or sensor API, or anything else you can ingest continuously. A large, curated list of free, keyless real-time sources is at github.com/conduktor/public-streaming-api. **Clear any self-proposed dataset with the lecturer before starting Phase 2.**

Your system will:

1. **Ingest** a continuous data stream into AWS.
2. **Process it in parallel** through two paths:
 - a **batch layer** that computes accurate, comprehensive views over the full history (using MapReduce and/or Spark);

- a **speed layer** that computes low-latency, incremental views over recent data (using stream processing with windowing).

3. **Store and visualise** results using AWS services.

4. **Auto-scale** the compute based on workload.

The architecture you build is a **Lambda architecture**: the batch layer gives correctness over all data, the speed layer gives freshness over recent data, and a serving layer answers queries by combining the two.

What You Will Build

A pipeline with these components:

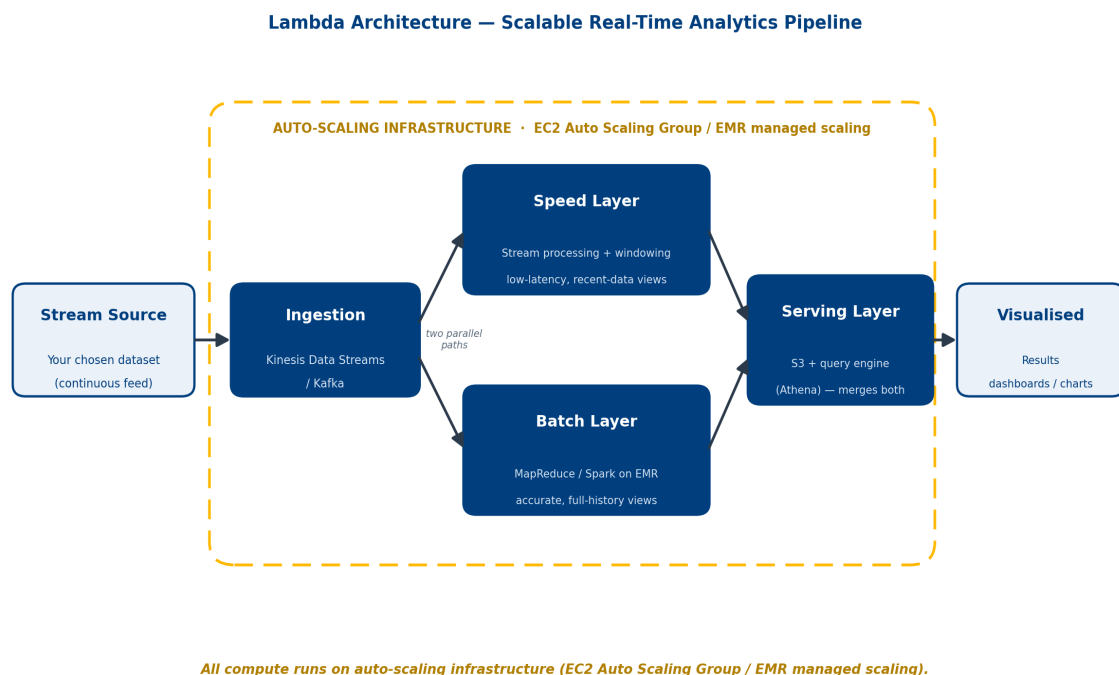


Figure 1 — Lambda architecture data flow for the scalable real-time analytics pipeline.

Phase 1: Design & Setup (20 Marks)

Problem Analysis & Use Case (5 Marks)

- Define the problem and the real-time question your system answers (e.g. "which keywords are trending in the last 5 minutes?", "which transit routes are most delayed right now?", "what is the rolling 1-minute volume-weighted average price?").
- Justify why a Lambda architecture (batch + speed) is appropriate, rather than batch-only or stream-only.

AWS Setup & Architecture Diagram (15 Marks)

- Design and present a clear architecture diagram showing ingestion, batch layer, speed layer, serving layer, and the auto-scaling boundary.
- Stand up the necessary AWS services (e.g. Kinesis/Kafka for ingestion, EMR or EC2 for processing, S3 + Athena for serving).
- Configure auto-scaling policies (an EC2 Auto Scaling Group with a target-tracking or step policy, or EMR managed scaling) and state the scaling triggers and cooldowns.

Data Ingestion (included in the above)

- Ingest your chosen stream into AWS using Kinesis Data Streams (or Kafka). You may implement the producer in Python (boto3) or Spark, or use a managed ingestion tool (e.g. Kinesis Data Firehose or the Kinesis Agent) — any approach is acceptable. Demonstrate records flowing into the system.

Phase 2: Parallel Processing (45 Marks)

Batch Layer — MapReduce / Spark (15 Marks)

- Implement the batch view over the full history using a Python-based MapReduce approach (Hadoop Streaming with Python scripts (PySpark on EMR)).
- Compute a meaningful aggregate appropriate to your use case (e.g. word count, keyword frequency, per-entity summarisation, sentiment aggregation, top N).
- Demonstrate correct, complete results over the accumulated dataset.

Speed Layer — Stream Processing (15 Marks)

- Implement real-time filtering and counting on the incoming stream (e.g. per-record processing in a Lambda function, short-interval aggregation, anomaly detection).
- Demonstrate low-latency incremental results that update as new data arrives.
- Implement **sliding-window** operations (e.g. "top 5 trending items in the last 5 minutes", "rolling 1-minute average", "count of events per window"). Windowed stream processing is the discriminator for distinction-level speed layers.

Hybrid Parallelism & Serving (15 Marks)

- Combine **data parallelism** (the batch MapReduce/Spark job) with **task parallelism** (distributing work across EC2 / EMR worker nodes).
- Produce a serving view that combines the batch and speed results (the Lambda "merge").
- **Benchmark** the system: compare sequential vs parallel execution of the batch job, and report how the speed layer's latency behaves under load.

Phase 3: Performance Measurement & Reporting (35 Marks)

Performance (10 Marks)

- Measure **throughput, latency, and speedup** under different loads (e.g. vary the ingestion rate, vary the number of worker nodes).
- Plot relevant graphs: speedup vs worker count, latency vs ingestion rate, throughput over time.

Report (15 Marks)

- Use the standard **IEEE double-column format** (≤ 10 pages).
- Include: objectives, architecture, tools, implementation, results, and a **critical analysis** (what scaled well, what bottlenecked, what you would change).

Demo Video (10 Marks)

- Prepare a video presentation walking through the system: the architecture, the live pipeline, the batch and speed views, the auto-scaling in action, and the benchmark results.

Marking Rubric

Phase	H1 (>70%)	H2.1 (>60%)	H2.2 (>50%)	Pass (>40%)	Fail (<40%)
Phase 1 — Use Case	Innovative use case; strong justification for the Lambda (batch + speed) split	Good use case; valid reasoning	Basic use case; some justification	Limited clarity; weak justification	No clear problem or justification
Phase 1 — Architecture & Setup	Complete cloud setup with clear Lambda diagram; auto-scaling correctly defined with stated triggers	Mostly complete AWS setup; some integration	Reasonable AWS structure; partially incomplete	Basic or incorrect setup; poor diagram	Missing or invalid architecture
Phase 1 — Ingestion	Seamless stream ingestion using Kinesis/Kafka + Python/Spark	Mostly functional ingestion	Working ingestion with limitations	Basic ingestion; lacks real-time handling	No working ingestion
Phase 2 — Batch Layer	Efficient MapReduce/Spark batch view; accurate, complete results	Mostly correct logic; minor inefficiencies	Working logic; some parallelism applied	Mostly sequential; basic logic	Broken or missing implementation
Phase 2 — Speed Layer	Sliding-window operations (e.g. top-N in last N minutes) implemented well; low-latency incremental views	Windowing attempted with minor bugs, or excellent real-time per-record processing	Working real-time stream logic (filtering/counting/anomaly detection)	Minimal streaming logic or delays	No stream processing implemented
Phase 2 — Hybrid & Serving	Combines batch + speed in a coherent serving view; data + task parallelism; clear benchmarks	Reasonable hybrid strategy	Partial hybrid; minimal benchmark	Weak hybrid logic; lacks comparison	No combination or evaluation
Phase 3 — Performance	Multiple metrics (latency, throughput, speedup); clear graphs and analysis	Good benchmarks and graphs; moderate analysis	Basic benchmarks and comparison	Minimal metrics; weak visuals	No benchmarking or invalid results
Phase 3 — Report	Well-formatted IEEE; structured; citations; high clarity	Mostly well written; few format/citation errors	Acceptable formatting and structure	Weak writing or poor formatting	Not in IEEE format or poorly written
Phase 3 — Demo Video	Clear, well-paced demo; covers all aspects; polished delivery	Good video; all key steps shown	Basic walkthrough; some missing pieces	Hard to follow or lacks coherence	Missing or not understandable